# 1) STUDY OF VARIOUS NETWORK COMMANDS USED IN LINUX AND WINDOWS

1) arp -a

=> Address resolution protocol

=> It show IP address and MAC address of router.

2) hostname:

=> Displays name of the computer

3) IPconfig /all:

=> Displays detailed configuration information about your TCP/IP connection including router, gateway, DNS, DHCP and type of ethernet adapter in your system.

4) nbstat -a:

=> command helps to solve problem with NetBIOS name resolution

5) netstat:

=> Display variety of statistics about a computers active TCP/ICP connections.

eg: netstat -n

6) nslookup:

=> DNS lookup in linux

=> Display DNS details, such as the IP address of a particular computer, the MX records for a domain or the NS servers of a domain.

eg: nslookup www.google.com

7) ping:

To test connectivity between two nodes.

1) ping hostname (ping localhost)

2) # ping id address (ping 4.2.2.2)

3) # ping fully qualified domain name (ping www.facebook.com)

2) Ifconfig

The ifconfig command was/is a staple in many sysadmin's tool belt for configuring and trouble shooting networks.

Ip:

This command combines the functionality of ping and traceroute commands.

a) TO show statistics, including each hop (hostname) with time and loss :).

```
# mtr google.com
```

b) TO show numeric IP address

```
# mtr -g google.com
```

c) show numeric IP address and host names

```
# mtr -b google.com
```

d) set the number of pings that you must to send

```
# mtr -c 10 google.com
```

IP:

The boss ip command can show address information, manipulate routing, plus display network various devices, interface and tunnels.

a) TO show the IP address assigned to an interface on your server

```
# ip address show
```

b) TO assign an IP to an interface,

for eg # ip address add 192.168.1.254/24 dev enp003

c) To delete an IP on an interface

# ip address del 192.168.1.254/24 dev enp503

d) Alter the status of the interface by the bringing the interface eth0 online.

# ip link set eth0 up

e) Alter the status of the interface by the bringing the interface eth0 offline.

# ip link set eth0 down

f) Alter the status of the interface enabling promiscuous mode for eth0:

# ip link set eth0 promisc on

g) Add a default route via the local gateway 192.168.1.254 that can attached on device eth0:

# ip route add default via 192.168.1.254 dev eth0

h) Add a route to 192.168.1.0/24 via the gateway at 192.168.1.254

# ip route add 192.168.1.0/24 via 192.168.1.254

i) Add a route to 192.168.1.0/24 that can be reached on device eth0.

```
# ip route add 192.168.1.0/24 dev
                    eth0
```

ii) Delete the route for 192.168.1.0/24 via the gateway at 192.168.1.254

```
# ip route delete 192.168.1.0/24 via
                    192.168.1254
```

tcpdump:

=> command for capturing and displaying packets.

a) TO install tcpdump

```
# dnf -y tcpdump
```

b) TO know which interfaces tcp dump can use

```
# tcpdump -D
```

c) TO capture traffic on eth0, we can initiate that with tcpdump -i eth0 sample, output:

```
# tcpdump -i eth0
```

d) # tcpdump -i eth0 -c 10 => capture traffic to and from one host.

e) # tcpdump -i eth0 -c 10 host 88.8.8
LL> TO filter out traffic coming from specific host

f) # tcpdump -i eth0 src host 8.8.8.8 => coming from 8.8.8.8

g) # tcpdump -i eth0 dst host 8.8.8.8
LL> outbound traffic going to 8.8.8.8

h) # tcpdump -i eth0 port 53
TO capture specific port.

i) # tcpdump -i eth0 -c 10 host www.google.com and port 443
TO capture only HTTPS path.c.

# 2) STUDY OF NETWORK CABELS CONFIGURING AN ETHERNET CONNECTION BY USING NMCII

## PROCEDURE

1) List the network manager connection profile:

#nmcii connection show

2) #nmcii connection add con-name < connection name > ifname < device-name > type ethernet.

3) optional: Rename the connection profile

#nmcii connection modify "wired connection 1"

4) Display the current settings of the connection

#nmcii connection show

5) Configure the IPV4 settings:

=> TO use DHCP, enter:

#nmcii connection modify "wired connection 1" ipv4.method auto

=> TO set static IPV4 address, network mask, default gateway, DNS servers, and search domain

enter:

#nmcli connection modify "wired connecti

ipv4.method manual ipv4.addrossess

192.0.2.1/24

ipv4.gateway 192.0.2.25 ipv4.dns

192.0.2.200

ipv4.dns-search example.com

6) configure the IP-v settings

To use stateless auto configuratio

enter:

#nmcli connection modify "wired connection"

ipv6.method auto

7) To activate the profile:

#nmcli connection up Internal-LAN:

# 3. BUILDING ANONYMOUS FTP SCANNER USING FTPLIB MODULE

## Aim:

To create a python-based FTP scanner using the ftplib module that:

* checks if a public FTP server allows anonymous sign
* Lists the contents of directories of the server

## Procedure:

* Import the built-in ftplib module
* Write two python scripts
* sender side: Connect to the FTP server, log in anonymously, list the root directory.
* Receiver side: connect anonymously and list a specific sub-directory
* Run the script in google colab or any python environment.
* observe the login status, server message and directy contents.

## Code:

```
from ftplib import FTP

# Get FTP server address from the user:
host = input("Enter FTP server address: ")

try:
```

```
# Create FTP connection
ftp = FTP()
print("In connecting to FTP server...")

# connect to the server
ftp.connect(host, 21, timeout=10)
print("connected to:", host)

# Try anonymous login
print("checking anonymous FTP Success...")
ftp.login(
        user="anonymous",
        passwd="anonymous@example.com"
)
# If login is successful
print("In [+] Anonymous FTP login is
        allowed")

# Display server welcome message
print("server response:", ftp.getwelcome())

# Display files/directories
print("In files and directories:")
ftp.dir()

# close the FTP connection
  ftp.quit()
except exception as e:
  print("In [-] Anonymous FTP login
        is not available.")
  print("Reason:", e)
```

Result:

Thus, the Python script using ftplib was successfully executed to check anonymous FTP login and display the server's directory contents.

# DEVELOP A SIMPLE CALCULATOR USING XML-RPC

**Aim:**

To develop a simple Calculat application using the XML-RPC Proto in Python that performs arithmeti operations through client-server communication.

**Code: server-side code (server.py)**

```
from xmlrpc.server import SimpleXMLRPCServ

class calculator:
    def add(self, x, y):
        return x+y

    def subtract(self, x, y):
        return x-y

    def multiply(self, x, y):
        return x*y

    def divide(self, x, y):
        if y != 0:
            return x/y

        else:
            return "Error: Division by zero is not allowed"
```

```
server = SimpleXMLRPCServer(("localhost", 8000))
print("XML-RPC server is running on
      port 8000...")
server.register_instance(Calculator())
server.serve_forever()
```

Code: Client-side code (client.py)

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy
        ("http://localhost:8000/")
print("Addition: 10+5 =", proxy.add(10,5))
print("Subtraction: 10-5 =", proxy.subtract
                              (10,5))
print("Multiplication: 10*5 =", proxy.
                              multiply(10,5))
print("Division: 10/5 =", proxy.divide(10,5))
print("Division by zero: 10/0 =",
      proxy.divide(10,0))
```

Procedure:

* Open a python IDE or terminal
* Create two separate python files: server.py & client.py
* Write the provided server-side code in server.py & client-side code in client.py
* Run the server.py and client.py file
* Observe the result displayed on the client screen & verify the server output.

OUTPUT:

Server-Side O/P: XML-RPC server is running on Port 8000

Client-Side O/P:

* Addition : 10+5=15
* Subtraction : 10-5=5
* Multiplication : 10*5=50
* Division : 10/5=2.0
* Division by zero: 10/0= error:

Result:

The XML-RPC based Calculator was successfully implemented. The client communicated with the server using XML-RPC and performed all arithmetic operations.

# DEVELOP A PROGRAM TO CREATE REVERSE SHELL USING TCP SOCKETS

**Aim:**

The aim of this exercise is to understand and implement a reverse shell using TCP sockets

**Procedure:**

* Create a new python script on the attacker machine
* Copy the server-side code

**Code: client.py (on the victim machine)**

```
import socket
client_socket = socket.socket(socket.
                AF_INET, socket.SOCK_
                STREAM).
server_ip = "127.0.0.1"
server_port = 9999
client_socket.connect((server_ip,
                       server_port))
print("connected to the server.")
print("type 'bye' to end the chat")
while true:
    message = input("client: ")
    client_socket.send(message.
                       encode())
```

```
if message.lower() == "bye":
    print("Chat ended.")
    break

reply = client_socket.recv(1024).decode()
print("Server:", reply)
if reply.lower() == "bye":
    print("Server ended the chat.")
    break

client_socket.close()
```

code: server.py (on the attacker machine)

```
import socket

server_socket = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM)

host = "127.0.0.1"
Port = 9999
server_socket.bind((host, Port))
server_socket.listen(1)
print("Server started...")
print("Waiting for a client to
        connect...")
client_socket, client_address
= server_socket.accept()
print("Client connected:", client_
        address)
```

```
while True:
    message = client_socket.recv(1024).
              decode()
    if not message:
        break
    print("Client:", message)
    if message.lower() == "bye":
        print("Client ended the chat.")
        break

client_socket.close()
server_socket.close()

print("Server stopped.")
```

Result:

A TCP client-server program was successfully created using python's socket module to establish a two-way connection & exchange messages with a local server on port 9999.

# DEVELOP A CUSTOMIZED PING COMMAND TO TEST SERVER CONNECTIVITY

**Aim:**

To test and verify the connectivity between the local computer & remote server using the ping command in Windows.

**Procedure:**

* open command prompt in Windows
* Type ping www.google.com and press Enter
* Type ping localhost to check the local TCP/IP stack
* Type ping 8.8.8.8 to test connectivity using an IP address.
* Type ping www.google.com -t for a continuous ping test. press ctrl+c to stop
* Type ping www.google.com -n 5 to send 5 echo requests
* Type ping www.google.com -l 1000 to test with a 1000-byte packet size

* Observe the reply, response time & packet loss displayed from each command
* Verify the connectivity between the local computer and remote server

Code:

```
import subprocess
import platform

def ping_server(host, count=4):
    """Ping a server and display the result."""
    if platform.system().lower() ==
                                "windows":
    else:
        command = ["ping", "-c",
                   str(count), host]
    try:
        result = subprocess.run(
            command,
            capture_output=True,
            text=True
        )
        print("\n--- Ping Results ---")
        print(result.stdout)
        if result.returncode == 0:
```

```
        print("\n [+] Server is
                reachable")
    else:
        print("\n [-] Server is
                not reachable")
except Exception as e:
    print(" Error:", e)
host = input(" Enter server IP
        address or domain name: ")
count = int(input(" Enter number of
        ping requests: "))
ping_server(host, count)
```

Result:

The Python script successfully executed the system os, determined host reachability using the process return code (0).